Paw Tracker

Dog Sport Tracker App
Sprint 4

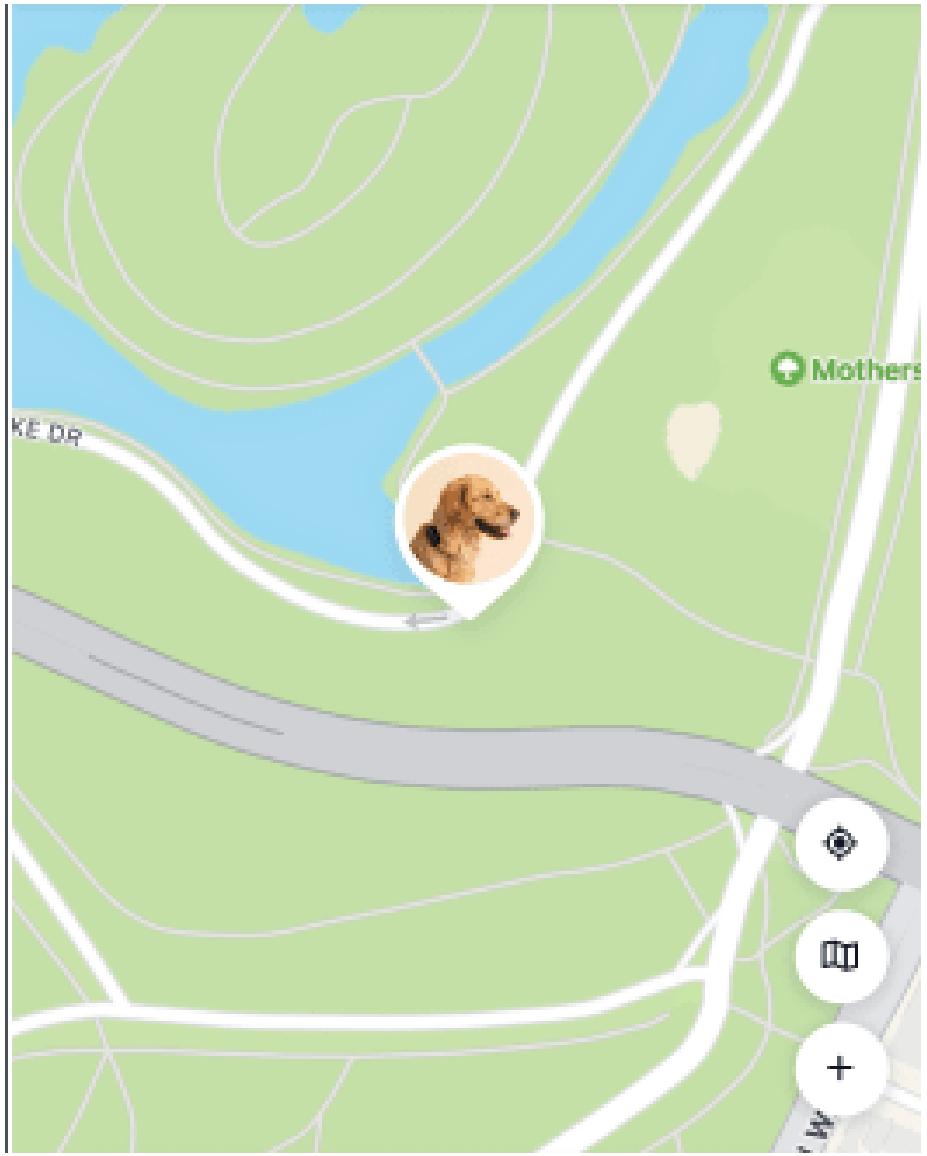
Mobile Application Development Project

Emmi Korhonen (Scrum Master)

Farnaz Jamilpanah

Sprint 4 Goals:

- Ui improvements:
  - Prepare UI for final presentation/demo.
  - Enhance visual hierarchy
  - Define more to color palette
  - Standardize typography styles
  - Improve welcome layout spacing
  - Add icons for breed, age, height and weight
  - Add subtle background or header image
  - Improve scroll behavior
  - Ensure all screens visually match the design system
- Navigation
- MainScreen and ProfileScreen
- Testing
  - Local unit tests, (UI tests, Room tests)



Jira:

Sprint 4 15 huhti – 22 huhti (8 työtehtävää) | 0 | 13 | 12 | Päätä sprintti

~~APP-4~~	Sprint-4-Planning	VALMIS	15.4	3
~~APP-61~~	As a new user, I want to create a dog profile so I can track my pet's activity	VALMIS		4
~~APP-104~~	As a new user, I want to see a main screen when I open the app for the first time so that I immediately understand what the app does	VALMIS		-
~~APP-9~~	Sprint-4-Retrospective	VALMIS	22.4	2
~~APP-58~~	Sprint-4-Review	VALMIS	22.4	3
APP-100	As a user, I want to navigate between different screens	KÄYNNISSÄ		2
APP-98	As a user, I want a consistent, clear and visually consistent interface, so that I can easily navigate the app, understand key information and interact with features with...	KÄYNNISSÄ		5
APP-99	As a user, I want the app to be tested, so that I can trust it works reliably	KÄYNNISSÄ		6

+ Luo

As a user, I want to navigate between different screens

Fix: Navigation between most screens is working correctly. However navigation behavior is not fully handled for MainScreen and to EditProfileScreen

Emmi’s tasks:

- **Testing**
  - Unit tests were implemented for StatisticsViewModel and HistoryViewModel
  - Tests included:
    - Ui state updates
    - Filtering logic (daily / weekly)
    - Repository interactions
    - Coroutine and Flow-based asynchronous behavior

Test Summary

4	0	0	1.116s
tests	failures	ignored	duration

100%
successful

Packages | Classes

Class	Tests	Failures	Ignored	Duration	Success rate
com.example.pawtracker.StatisticsViewModelTest	1	0	0	1.029s	100%
com.example.pawtracker.ui.history.HistoryViewModelTest	3	0	0	0.087s	100%

HistoryViewModelTest

- Verified filtering logic for daily and weekly walks
- Correct UI state updates from Flow-based repository data

- Fake repositories
- Kotlin Coroutines Test, advanceUntilIdle
- StateFlow testing
- Flow testing Turbine / awaitItem

```kotlin
@Test
fun `loads walks and filter daily correctly`() = runTest {

    val repo = FakeWalkRepository(
        walks = listOf(
            walkToday(),
            walkYesterday()
        )
    )

    val viewModel = HistoryViewModel( repository = repo)

    advanceUntilIdle()

    viewModel.uiState.test {
        val state = awaitItem()

        assertEquals( expected = 1,  actual = state.walks.size)
    }
}
```

```kotlin
@Test
fun `filters weekly correctly`() = runTest {

    val repo = FakeWalkRepository(
        walks = listOf(
            walkToday(),
            walkThisWeek(),
            walkLastWeek()
        )
    )

    val viewModel = HistoryViewModel( repository = repo)

    advanceUntilIdle()

    viewModel.setFilter(WalkFilter.Weekly)

    viewModel.uiState.test {
        val state = awaitItem()

        assertEquals( expected = 2,  actual = state.walks.size)
    }
}
```

```kotlin
@Test
fun `filter change updates ui state`() = runTest {

    val repo = FakeWalkRepository(
        walks = listOf(walkToday(), walkThisWeek())
    )

    val viewModel = HistoryViewModel( repository = repo)

    advanceUntilIdle()

    viewModel.setFilter(WalkFilter.Weekly)

    val state = viewModel.uiState.value

    assertEquals( expected = WalkFilter.Weekly,  actual = state.filter)
}
```

```kotlin
// Helper data
3 Usages
private fun walkToday(): WalkEntity {
    return WalkEntity(
        id = 1,
        startTime = now(),
        endTime = now(),
        distance = 1f,
        duration = 100L,
        pointCount = 10
    )
}
```

StatisticsViewModelTest

```
@Test
fun `combines repository data into statistics ui state`() = runTest {

    val viewModel = StatisticsViewModel(
        walkRepository = FakeWalkRepository(),
        profileRepository = FakeDogProfileRepository()
    )

    viewModel.uiState.test {

        awaitItem()
        val state = awaitItem()

        advanceUntilIdle()

        assertEquals( expected = 2f,  actual = state.todayDistance)
        assertEquals( expected = 100L,  actual = state.todayDuration)
        assertEquals( expected = 10f,  actual = state.weekDistance)
        assertEquals( expected = 500L,  actual = state.weekDuration)

        assertEquals( expected = 5f,  actual = state.goalDistance)
        assertEquals( expected = 30L,  actual = state.goalDuration)

        assertEquals( expected = "Nelli",  actual = state.dogName)
        assertEquals( expected = "testUri",  actual = state.imageUri)
```

```
@Test
fun `filter change updates ui state`() = runTest {

    val repo = FakeWalkRepository(
        walks = listOf(walkToday(), walkThisWeek())
    )

    val viewModel = HistoryViewModel( repository = repo)

    advanceUntilIdle()

    viewModel.setFilter(WalkFilter.Weekly)

    val state = viewModel.uiState.value

    assertEquals( expected = WalkFilter.Weekly,  actual = state.filter)
}
```

FakeRepositories

- Used to model database behavior in tests

```
5 Usages
class FakeWalkRepository( private val walks: List<WalkEntity> = emptyList()) : WalkRepository {

    override fun getTodayDistance() = flowOf( value = 2f)
    override fun getTodayDuration() = flowOf( value = 100L)
    override fun getWeekDistance() = flowOf( value = 10f)
    override fun getWeekDuration() = flowOf( value = 500L)

    override fun getAllWalks() = flowOf( value = walks)

    override fun getTodayWalks() = flowOf( value = walks)

    override fun getWalksByWeek() = flowOf( value = walks)

    override suspend fun getWalkDetails(walkId: Long): WalkWithPoints {
        return WalkWithPoints(

            walk = walks.firstOrNull() ?: WalkEntity(
                id = walkId,
                startTime = 0L,
                endTime = 0L,
                distance = 0f,
                duration = 0L,
                pointCount = 0
            ),
            points = emptyList()
        )
    }

    override suspend fun insertWalkWithPoints(
        walk: WalkEntity,
        points: List<LocationPoint>
    ) {

    }
    override suspend fun deleteWalk(walk: WalkEntity) {
    }
}
```

```
1 Usage
class FakeDogProfileRepository : DogProfileRepository {

    override fun getProfile() = flowOf(
        value = DogProfileEntity(
            name = "Nelli",
            imageUri = "testUri",
            dailyDistanceGoal = 5f,
            dailyDurationGoal = 30L,
            weeklyDistanceGoal = 20f,
            weeklyDurationGoal = 120L
        )
    )

    override suspend fun saveProfile(profile: DogProfileEntity) {}
}
```

Farnaz’s tasks:

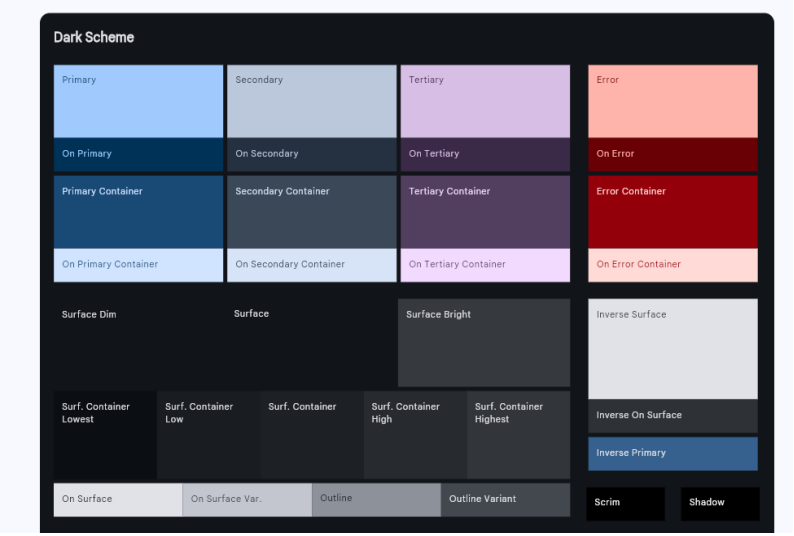
- MainScreen, ProfileScreen
- Navigation, NavBar
- Ui improvements

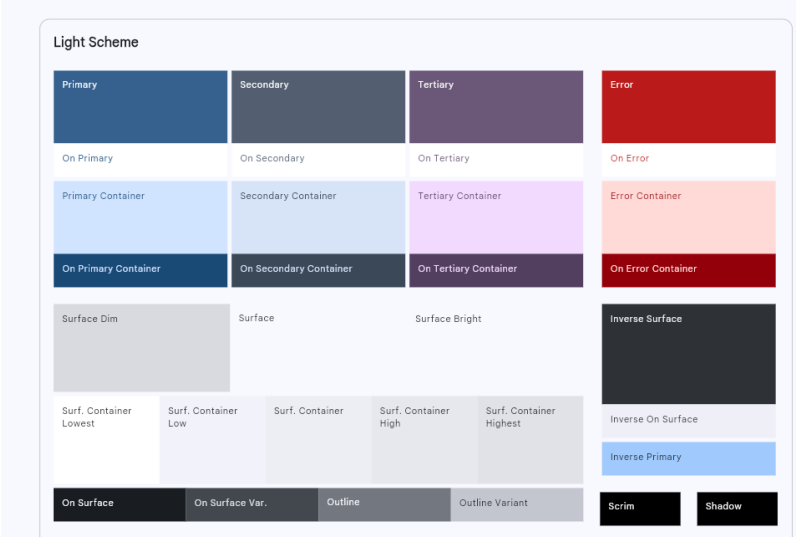
Material 3Theming:

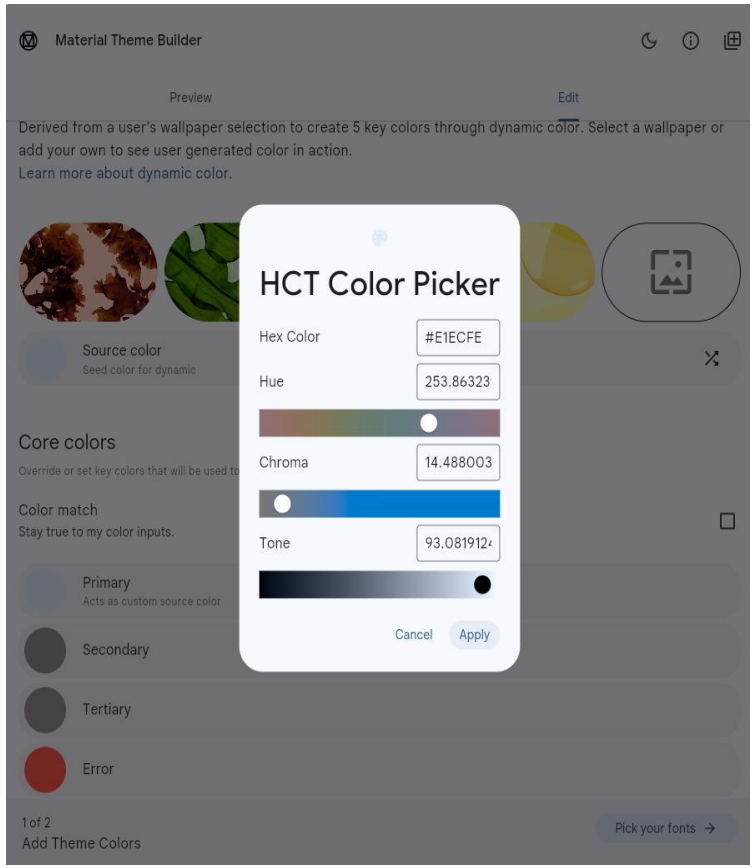
A full Material 3 color system

Generated from Material Theme Builder:

Material 3 theme comprises the following subsystems to add theming to app: color scheme and typography. When you customize these values, your changes are automatically reflected in the M3 components used to build app.







Navigation Bar:

Screens.kt:
Defines routes

```
package com.example.pawtracker.ui.navigation
//Define routes:
21 Usages  5 Inheritors
sealed class Screen(val route: String) {
    4 Usages
    object Main : Screen( route = "main")
    4 Usages
    object Tracking : Screen( route = "tracking")
    3 Usages
    object History : Screen( route = "history")
    1 Usage
    object Statistics : Screen( route = "statistics")
    3 Usages
    object Profile : Screen( route = "profile")
}
```

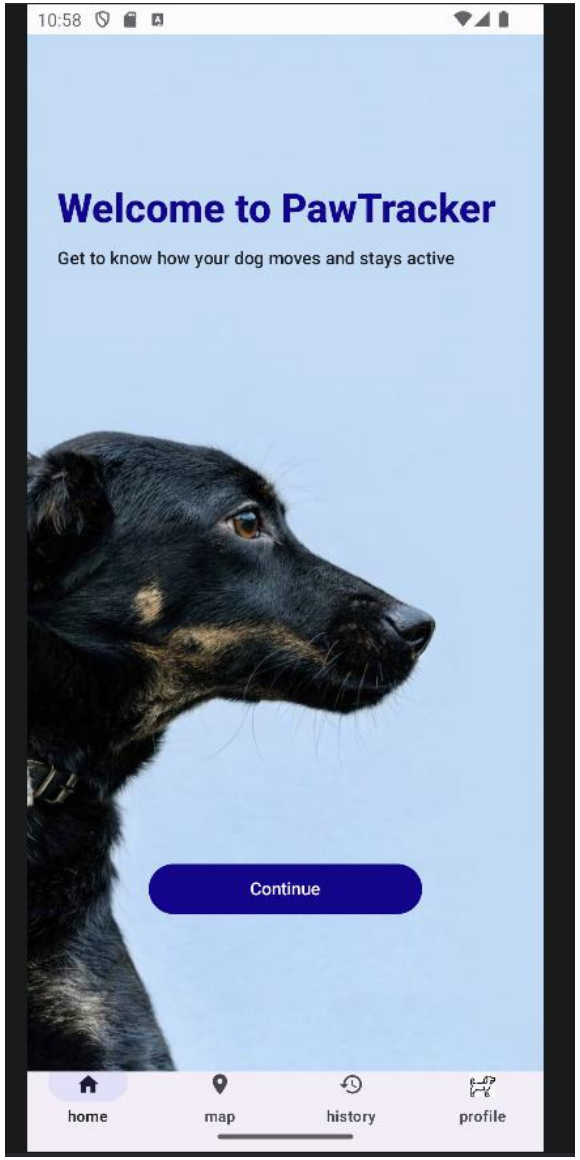
NavGraph:Navigate
to composable

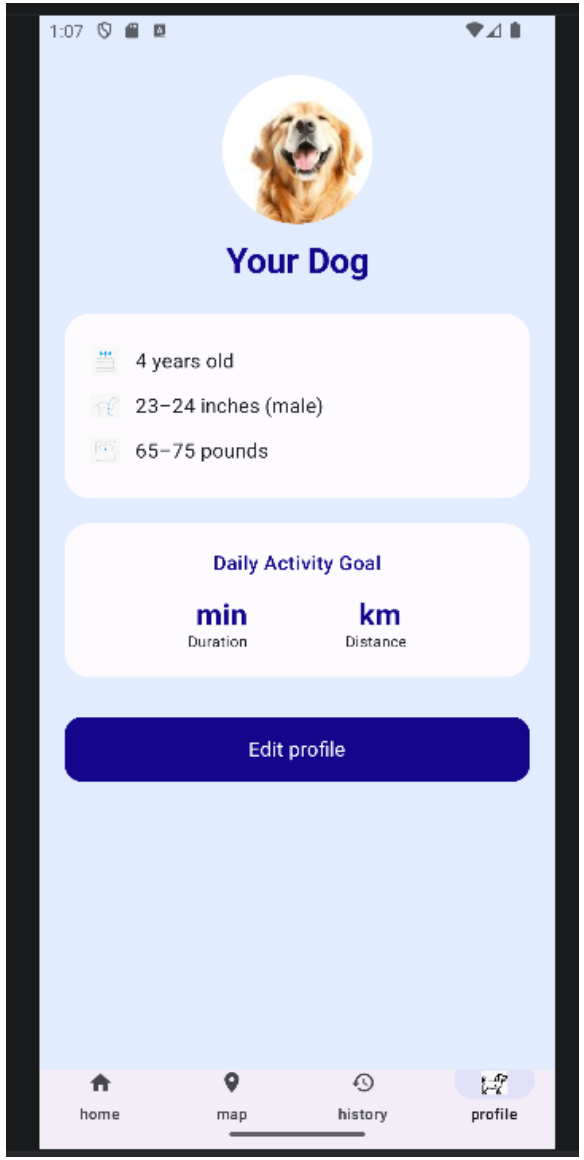
```
2 Usages
@Composable
fun NavGraph(navController: NavHostController,
             isDarkTheme: Boolean,
             onToggleTheme: () -> Unit,
             innerPadding: PaddingValues
) {
    NavHost(
        navController = navController,
        startDestination = Screen.Main.route
    ) {
        //Navigate to a composable
        composable(Screen.Main.route) {MainScreen( onContinueClick = {
            navController.navigate(Screen.Tracking.route)
        },
            isDarkTheme = isDarkTheme,
            onToggleTheme = onToggleTheme,
            innerPadding = innerPadding) }
        composable(Screen.Tracking.route) {TrackingScreen(innerPadding) }
        composable(Screen.History.route) {HistoryScreen(innerPadding) }
        composable(Screen.Statistics.route) {StatisticsScreen(innerPadding)}
        composable(Screen.Profile.route) { ProfileScreen(innerPadding = innerPadding) }
    }
}
```

NavBar.kt:

```
NavigationBarItem(
    selected = currentRoute == Screen.History.route,
    onClick = {
        navController.navigate(Screen.History.route) {
            popUpTo(navController.graph.startDestinationId) { saveState =
            launchSingleTop = true
            restoreState = true
        }
    },
    icon = { Icon( imageVector = Icons.Filled.History, contentDescription =
        modifier = Modifier
            .size(24.dp)
            .padding(top = 4.dp)) },
    label = { Text("history") }
)
```

MainScreen & ProfileScreen UI design:

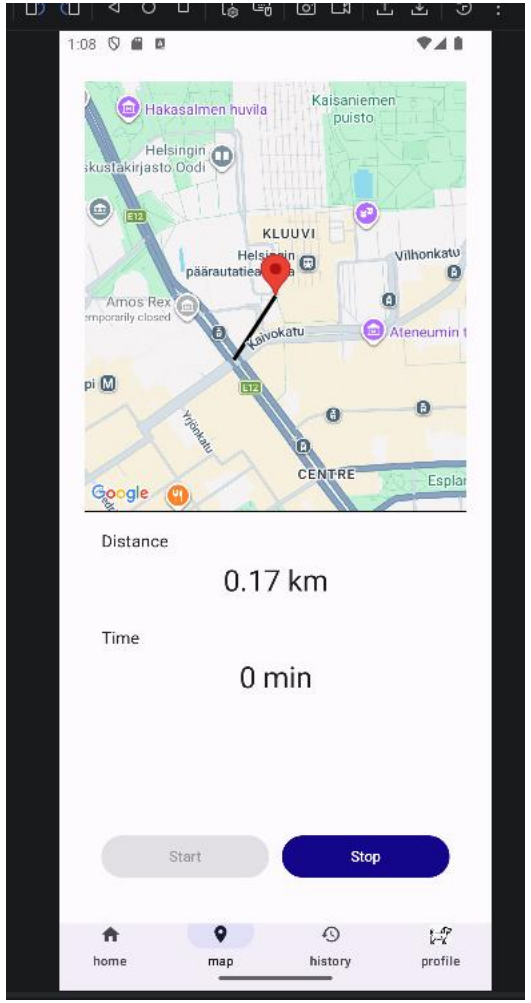


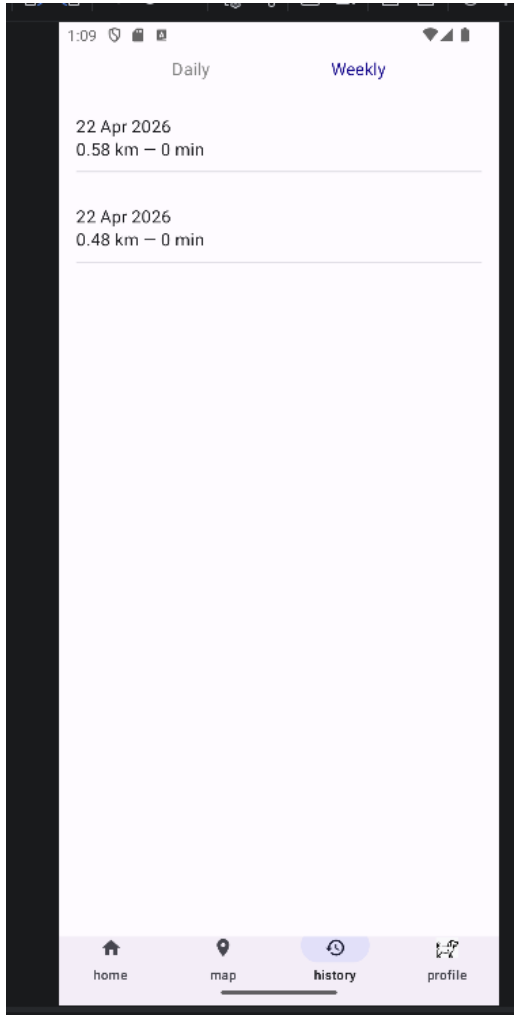


Implement Navbar for each Screen:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    // Ask for permission
    requestLocationPermission()
    enableEdgeToEdge()
    setContent {
        val systemDark = isSystemInDarkTheme()
        var isDarkTheme by rememberSaveable { mutableStateOf( value = systemDark) }

        PawTrackerTheme(darkTheme = isDarkTheme) {
            val navController = rememberNavController()

            Scaffold(
                bottomBar = { NavBar(navController) }
            ) { innerPadding ->

                NavGraph(
                    navController = navController,
                    isDarkTheme = isDarkTheme,
                    onToggleTheme = {isDarkTheme = !isDarkTheme },
                    innerPadding = innerPadding
```

```
@Composable
fun TrackingLayout(
    uiState: TrackingUiState,
    onStart: () -> Unit,
    onStop: () -> Unit,
    innerPadding: PaddingValues
) {
    Column(
```





Challenges:

app **stops immediately** when you tap a bottom-bar icon(There was **silent crash**) :

How to fix:

1. **missing typography:**use a typography style that does not exist
2. **Each Screens follows the Scaffold padding:**

put innerpadding as parameter for each screen and scaffold just in mainActivity

3. **Create viewmodelFactory for each screen:** A factory is simply a **helper** that tells Android *how to create your ViewModel* when the ViewModel needs constructor parameters

```
Column(
    modifier = Modifier
        .fillMaxWidth()
        .padding( all = 16.dp)
) {
    Text(walk.date, style = MaterialTheme.typography.titleMedium)
    Text("%.2f km — %s".format( ...args = walk.distanceKm, timeFormatted))
```

```
3 Usages
@Composable
fun TrackingScreen(innerPadding: PaddingValues) {
    val context = LocalContext.current

    val db = AppDatabase.getDatabase(context)

    val gpsRepository = GPSRepository(context)
    val walkRepository = WalkRepositoryImpl( dao = db.walkDao())

    // 3. Create ViewModel with factory
    val viewModel: TrackingViewModel = viewModel(
        factory = TrackingViewModelFactory(gpsRepository, walkRepository)
    )

    val uiState by viewModel.uiState.collectAsState()
```

Retrospective

Challenges:

- **Time management:**
  - Time allocation was more limited during this sprint

- **Testing:**
  - Implementing tests helped identify areas that could be optimized
  - The use of Flow and coroutines in ViewModels added complexity in testing

Sprint 5 goals:

- **Work that continues into next sprint:**
  - Testing
  - Ui improvements
  - Fix: Navigation for mainscreen and editprofilescreen

- **Focus: Testing & Bug Fixing**
  - Implementation of more unit tests
  - Bug fixing for existing features

Thank you!

We appreciate feedback